

Qoder Agent SDK 1.0.31 Upgrade Assessment

Date: 2026-09-01

Application: `opencode-qoder-bridge`

Bridge release: `0.1.11`

SDK release: `@qoder-ai/qoder-agent-sdk@1.0.31`

Recommendation

Keep the upgrade. It is worth adopting because the bridge now uses the SDK's native memory, security-scan, MCP OAuth, and session-control capabilities while preserving the existing OpenCode provider contract. The upgrade is opt-in for the features that can add latency or Qoder usage.

The SDK `1.0.31` package bundles `qodercli 1.1.38`. The upgrade therefore also brings the bundled CLI fixes for MCP authentication state and oversized-image compaction. If a separately installed `qodercli` is found first on `PATH`, that installation should be updated separately so it does not mask the bundled runtime.

What changed in the application

1. SDK and package alignment

- Pinned `@qoder-ai/qoder-agent-sdk` to `1.0.31`.
- Bumped the bridge package to `0.1.11`.
- Kept the SDK as a direct dependency; no SDK code is vendored or patched.
- Added the SDK's `memory` and `securityScan` option types to the public bridge configuration.

2. Native memory and skill evolution

Memory is still disabled by default. Applications can enable project or user memory through the provider configuration:

```
{
  "provider": {
    "qoder": {
      "options": {
        "memory": {
          "mode": "native",
          "projectScope": true,
          "userScope": false
        }
      }
    }
  }
}
```

After a successful turn, the bridge gives Qoder's memory and skill-evolution background work up to 10 seconds to finish. Slow or failed background work is non-fatal and is logged only in debug mode, so it does not turn a successful model response into an application error.

Reference: [Qoder SDK memory](#).

3. Opt-in security scanning

Security scanning can be enabled without changing the current permission policy:

```
{
  "securityScan": {
    "l1StaticCheck": true,
    "l2LightweightScan": true,
    "l3DeepScan": false
  }
}
```

L1 runs after supported edits. L2 and L3 enable repository-level checks. These checks can add latency and consume Qoder credits, so production environments should enable them deliberately.

Reference: [Qoder SDK security scan](#).

4. MCP health and OAuth recovery

The bridge now exposes:

- `qoder_mcp_status` to inspect connection state, tool counts, and auth state.
- `qoder_mcp_auth` to start or complete the active MCP OAuth flow.

The OAuth flow is two-step: request an authorization URL, open it in a browser, then submit the complete callback URL. The bridge keeps the initialized SDK query alive for this flow and expires the pending state after 10 minutes.

Reference: [Qoder SDK MCP support](#).

5. Local session forks

`qoder_session_fork` creates an independent local transcript branch without changing the active provider-to-session mapping. The returned session ID can be continued explicitly when a parallel line of work is needed.

This is intentionally local to the bridge. A production shared session store or daemon was not enabled because this application does not provide a remote storage backend for it.

Automatic OpenCode slash commands

The plugin registers these commands in OpenCode's local TUI command layer and automatically adds its TUI entry to the user's `~/.config/opencode/tui.json`. Users do not need to edit `opencode.json` or `tui.json` manually. Use the regular TUI; the current `--mini` interface does not load external TUI plugins.

Command	Arguments	Purpose
<code>/qoder_usage</code>	none	Show live quota and local cost/token totals.
<code>/qoder_models</code>	none	List available models and capabilities.
<code>/qoder_sessions</code>	optional directory and/or limit	List recent sessions.
<code>/qoder_session_reset</code>	optional key, or <code>all</code>	Reset persisted sessions.
<code>/qoder_session_fork</code>	optional session details	Create a session branch.
<code>/qoder_mcp_status</code>	none	Inspect MCP connection and OAuth state.
<code>/qoder_mcp_auth</code>	server and optional callback URL	Start or complete MCP OAuth.
<code>/qoder_plan_mode</code>	none	Show Plan Mode guidance.

These are local display commands. Selecting one from the `/` autocomplete list runs its handler directly, opens an argument box when needed, and displays the result in a modal box. No LLM turn or model tokens are used. The matching Qoder tools remain available separately when an agent needs to call them.

Configuration guidance

Start with the default configuration. Enable features by environment:

Environment	Recommended options	Reason
Local development	Memory, optional L1 scan	Better project continuity with low overhead
Trusted CI	L1 and L2 scans	Catch common issues after repository edits
Sensitive repositories	L1, L2, and selective L3	More coverage; expect more latency and usage
Unauthenticated or offline use	Defaults	Native features require Qoder runtime access

Do not put Qoder PATs, OAuth callback URLs, or npm tokens in source files or documentation. Use the existing environment and Qoder login flows.

Validation completed

The bridge validation command passed:

```
npm run typecheck
npm test
146 passed, 2 intentional skips, 0 failures
npm run test:stress
13 passed, 1 intentional skip, 0 failures
npm run audit
0 npm audit vulnerabilities
```

The check covers the SDK option mapping, SDK 1.0.31 exports, MCP tools, session forks, streaming behavior, Windows path handling, and existing bridge regressions.

Upgrade checklist

1. Install the bridge with the lockfile so SDK 1.0.31 is selected.
2. Restart OpenCode after installation so the provider and model catalog reload.
3. Run `qoder_mcp_status` if MCP servers are configured.
4. Enable memory or scanning only after deciding the desired scope and usage budget.
5. If an external `qodercli` is selected from `PATH`, update it through the normal Qoder CLI installer as well.

References

- [Qoder Agent SDK TypeScript reference](#)
- [Qoder SDK memory](#)
- [Qoder SDK security scan](#)
- [Qoder SDK MCP](#)
- [Qoder CLI update log](#)